Project-01: Building Scalable, Secure, and High-Performance Web Applications with AWS 3-Tier Architecture

Project Overview
The primary objective of this project is to deploy a scalable, highly available, and secure python application using a 3-tier architecture on Amazon Web Services (AWS). The solution should leverage AWS services such as Amazon EC2, Amazon RDS, Amazon VPC, Application Load Balancer, and Auto Scaling to ensure optimal performance, fault tolerance, security, and availability.

Repository Link: https://github.com/Vennilavanguvi/Three-Tier-Architecture.git

Task 1 : Design and Configure the AWS Networking Infrastructure

Create the networking components required for the application.

Tasks

Create the following AWS resources:

- Custom VPC
- Internet Gateway
- NAT Gateway
- Elastic IP
- Route Tables
- Subnet Associations

VPC

Create a VPC using an appropriate CIDR block.

Subnets

Create 6 Subnets distributed across 2 Availability Zones.

Subnet Type	Quantity
Public (Web Tier)	2
Private (Application Tier)	2
Private (Database Tier)	2

Each subnet should support at least 256 IP addresses.

Internet Gateway

- Create an Internet Gateway.
- Attach it to the VPC.

NAT Gateway

- Allocate an Elastic IP.
- Create a NAT Gateway in one of the Public Subnets.
- Ensure private resources can access the Internet through the NAT Gateway.

Route Tables

Create 3 Route Tables.

Route Table	Associated Subnets
Public Route Table	Public Subnets
Private Application Route Table	Application Subnets

Private Database Route Table	Database Subnets

Configure appropriate routes and subnet associations.

Task 2 - Deploy the Web Tier (Presentation Layer)

Deploy the Presentation Layer.

Tasks

- Launch EC2 instances in the Public Subnets.
- Configure the required Security Group.
- Connect to the instance.
- Install Nginx.
- Copy the provided HTML files.
- Verify the web page is accessible.

Task 3 - Deploy the Application Tier (Backend API)

Deploy the Backend API.

Tasks

- Launch EC2 instances in the Private Application Subnets.
- Connect through the Bastion Host (if used) or Session Manager.
- Install Python 3.
- Install Flask and required Python packages.
- Check App.md in github for instructions.
- Copy app.py file from given repository.
- Install dependencies.
- Run the Flask application on Port 5000.

- Verify the application is running.

Task 4 - Deploy and Configure the Database Tier

Deploy the Database Layer.

Tasks

- Create an Amazon RDS MySQL instance.
- Deploy the database in the Database Subnets.
- Configure database credentials.
- Check DB.md in Github for instructions.
- Connect DB in App Ec2 terminal.
- Execute the provided SQL commands in the terminal.
- Verify connectivity from the Flask application.

Task 5 - Implement Network and Instance Security

Create Security Groups with appropriate inbound and outbound rules.

Configure Security Groups for:

- Web Tier
- Application Tier
- Database Tier
- Load Balancer

Ensure only the required ports are allowed between tiers.

Task 6 - Configure the Internet-Facing Application Load Balancer

Create an internet-facing Application Load Balancer.

Tasks

- Create a Target Group.
- Register Web EC2 instances.
- Configure Health Checks.
- Attach the Load Balancer to the Public Subnets.
- Verify the application is accessible using the ALB DNS name.

Task 7 - Configure Internal Application Load Balancer

Create an Internal Load Balancer for the Application Tier.

Tasks

- Create an Internal ALB.
- Create a Target Group.
- Register Flask EC2 instances.
- Configure Health Checks.
- Verify communication between the Web Tier and Application Tier.

Task 8 - Configure Nginx Reverse Proxy for API Routing

Update the Nginx configuration.

Tasks

- Replace the default Nginx configuration with the provided `nginx.conf`.

- Copy nginx.conf file from given github and replace. Check proxy.md in github.
- Configure Nginx to forward API requests to the Internal Application Load Balancer.
- Restart the Nginx service.
- Verify successful communication between the Web Tier and Application Tier.

Task 9 - Integrate the Flask Application with Amazon RDS

Update the application configuration.

Tasks

- Configure database connection details.
- Verify the application connects successfully to Amazon RDS.
- Submit sample feedback through the application.
- Confirm the data is stored in the database.

Task 10 - Validate the Complete 3-Tier Architecture

Verify the complete application.

The learner should demonstrate:

- Web application is accessible through the Public Application Load Balancer.
- Feedback form accepts user input.
- Nginx successfully forwards API requests to the Internal Load Balancer.
- Flask application processes requests successfully.
- Feedback data is stored in Amazon RDS MySQL.
- Data can be verified using SQL queries.
- Security groups allow only the required traffic between application tiers.

Additional Task (Bonus): Configure Auto Scaling

Enhance the architecture by implementing **Amazon EC2 Auto Scaling** for improved scalability and fault tolerance.

Submission Deliverables

- Working Application Link (Public ALB DNS) Screenshot
- PDF/Word document with screenshots for each project step
- AWS Architecture Diagram
- SQL query output showing stored feedback records